

PKI Audit Platform

Quality Assurance & Security Blueprint

Document Version: 1.0 | Status: Active Development

This document serves as the formal record of testing methodologies applied to the PKI Audit Platform to date, and establishes the required acceptance criteria and security countermeasures for all upcoming architectural phases. It ensures a deterministic, highly resilient software lifecycle.

1. Current State: Completed Phases

COMPLETED

The core business logic and data access layers are complete and mathematically verified through unit testing.

Phase 1: Data Access & Infrastructure

- **Dependency Injection:** Core architecture utilizes interface-driven design (Repositories vs. Implementations).
- **Data Serialization:** Confirmed successful JSONB mapping for extended certificate traits within PostgreSQL.
- **Audit Logging:** Implemented and verified isolated transaction logging for state changes.

Phase 2: Service Layer & Business Logic (Nis2BaselineValidator)

The deterministic validation engine (`Nis2BaselineValidator`) has been subjected to rigorous mathematical and adversarial testing.

Testing Methodology	Focus Area	Verification Result
Happy Path	Standard compliant certificate (2048-bit RSA, SHA-256).	Verified system computes clean <code>AuditReport</code> .
Boundary Value Analysis	Maximum Lifespan (398 days). Warning Window (30 days). RSA Key Size limits (2048 bits).	Verified edges: 397 (Pass), 398 (Pass), 399 (Fail). 29, 30, 31 (Warnings mapped accurately).

Testing Methodology	Focus Area	Verification Result
Domain Rule Exemptions	Elliptic Curve Cryptography (ECC).	Verified 256-bit ECDSA bypasses legacy 2048-bit RSA limits.
Adversarial / Poison Pill	Null Master Shield, Null Timestamps, Missing Signature Algorithms.	Verified system gracefully catches <code>NullPointerException</code> vectors and logs critical violations rather than crashing JVM threads.

2. Upcoming: Phase 3 (API / Controller Layer)

IN PROGRESS

As the application is exposed to network traffic, the API layer must act as a strict defense gateway. The following Threat Models and defenses must be implemented before Phase 3 is considered complete.

Threat Model: DoS via Memory Exhaustion

Vector: Malicious actors submit massive payloads (e.g., 10GB+ arrays) mimicking `.pem` files, causing the JVM to hit an `OutOfMemoryError`.

Required Countermeasure: The API Controller must enforce a hard byte-size limit on incoming requests (e.g., max 25 KB). Over-limit payloads must be rejected immediately with HTTP `413 Payload Too Large` before reaching the Service Layer.

Threat Model: Infinite Loop / Zip Bomb

Vector: Corrupted X.509 chains designed to trap native parsing libraries in cyclic loops, exhausting server threads.

Required Countermeasure: Certificate parsing operations must be wrapped in a strict execution timeout. If parsing exceeds 2000ms, the operation must be aborted, returning HTTP `408 Request Timeout`.

Threat Model: Cross-Site Scripting (XSS) via Extracted SANs

Vector: A technically valid certificate containing executable scripts within the Subject Alternative Name (SAN) or Subject string, designed to execute when viewed in the Phase 5 React Dashboard.

Required Countermeasure: All strings extracted by the `CertificateExtractor` must pass through an HTML-sanitization routine before database persistence.

Phase 3 Acceptance Criteria:

- REST API routes successfully map standard HTTP Verbs (POST, GET).
- Integration tests (E2E) prove HTTP responses align with domain logic (e.g., compliant uploads return `200 OK`, invalid formats return `400 Bad Request`).

3. Upcoming: Phase 4 & 5 (Automation & Presentation)

PENDING

Phase 4: Automation Engine (Cron Worker)

The worker must run entirely decoupled from user interactions. Required testing includes:

- **Memory Leak Testing:** Verifying the cron job gracefully clears hibernate/JPA sessions after large batch queries to prevent progressive memory exhaustion.
- **State Lock Mitigation:** Implementing basic locking mechanisms to ensure two concurrent background jobs do not audit the same certificate simultaneously.

Phase 5: Frontend Dashboard

The presentation layer will visualize the APIs. Core security and QA requirements:

- Strict CORS configuration on the Java API to only accept requests from authorized frontend domains.
- Secure HTTP/TLS transit for all communications.
- Graceful UI handling of backend timeouts and 500 errors.